

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Design, brute-force validation, and a controlled comparison with Q3C and pgSphere

J. Salgado¹  and M. Delli Veneri¹ 

SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield SK11 9FT, United Kingdom
jesus.salgado@skao.int, michele.delliveneri@skao.int

Received ...; accepted ...

ABSTRACT

Context. Astronomical archives answer positional queries from relational databases, and in the PostgreSQL world two extensions dominate: Q3C, which keys each source by an integer drawn from a quad-tree on a cube and stores it in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree. The first is compact and fast for points; the second is general and indexes regions as well.

Aims. We ask whether the two can be combined: a single index that keeps the integer key and the B-tree, yet answers cones, polygons, cross-matches and stored footprints, and that speaks the geometry language archives already use. We aim to establish where such an index helps, where it does not, and what its cost model actually depends on.

Methods. We present **skycell**, a PostgreSQL extension that keys positions by their order-29 HEALPix cell in the nested scheme and turns a query region into index ranges. How finely to cut is decided by a cost model – the price of an extra index range against the rows it removes – with the local source density read from the histogram **ANALYZE** already keeps for the key column, which is a density map precisely because HEALPix cells are equal area. Cells are classified conservatively against the region, and a planner support function rewrites the predicate into B-tree range conditions plus an exact test. Coverings are validated by brute force against millions of catalogue points and against points drawn inside adversarial cones and polygons. Performance is measured against Q3C 2.0.5 and pgSphere 1.5.2 on two 10-million-row corpora – one synthetic, one resampled from the real Gaia DR3 density field – in trials where all three methods answer the same query back to back in a randomized order, repeated, analysed paired, warm and with caches dropped, and repeated again on a 50-million-row catalogue where pgSphere’s index no longer fits in memory.

Results. Warm, skycell is faster than pgSphere by 6–11% for cone radii of $1''$ – $1'$ and by 31–37% at 3° , and the two are indistinguishable between $6'$ and 1° ; on the Gaia field skycell is 3% slower at $6'$. With caches dropped it is faster at every radius on both corpora (ratios 0.39–0.95), touching a third of pgSphere’s buffer pages at 1° . It keeps an index of 214 MB against pgSphere’s 685 MB – 22.5 bytes per row against 72, which at Gaia DR3’s 1.8×10^9 sources is 38 GiB against the 160 GiB of the pgSphere-plus-Q3C pair an archive needs today to cover both cone searches and cross-matching; on a machine that holds the first and not the second, warm skycell runs 10–20 times faster than cold pgSphere – builds it in 1.4 s against 42 s, answers convex polygons 2.7–4.4 times faster, matches **q3c_join** on cross-matching – the operation Q3C is built for – while being 2.5–3.3 times faster than pgSphere at it, and its row estimates are wrong by a factor 1.2 rather than 2.5–3.0. The cost model’s one parameter is derived from the relation’s own statistics and the planner’s cost factors rather than configured, its chosen cell order costs about 3% against the best available one, and its density estimate is unbiased wherever a density is defined at all. Raising the catalogue to 50 million rows separates the two halves of the result: for regions of $30'$ and larger the advantage grows (at 1° , 0.85 to 0.60 warm and 0.53 to 0.31 cold), while for cones of an arcminute and below it does not move, because there both methods touch a handful of pages at any table size and what is left is planning. An ObsCore-shaped relation measured at three sizes puts the crossover with pgSphere between 0.5 and 2 million rows for degree-scale cones, and beyond 10 million for small cones on field-clustered data.

Conclusions. A cost-based covering over an equal-area integer key is a sound basis for a sky index, and the right question is not whether it is faster but for which queries. Its cost is paid per query and its benefit per row scanned, so it wins where enough rows are scanned to amortise a covering – degree-scale regions, polygons, cross-matching, cold caches – and that boundary moves up in radius as the catalogue grows denser. On a Gaia-sized catalogue a small cone search is not the workload this index improves.

Key words. Astronomical databases: miscellaneous – Catalogs – Virtual observatory tools – Methods: data analysis

1 Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory

these questions arrive as ADQL (Ortiz et al. 2008) through the Table Access Protocol (Dowler et al. 2019), and the service has to turn them into something a relational database can answer with an index.

Two PostgreSQL extensions carry most of this traffic. Q3C (Koposov & Bartunov 2006) projects

the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve; each source is one 64-bit integer in a B-tree, and the table can be sorted along the same curve so that neighbours on the sky are neighbours on disk. pgSphere (Chilingarian et al. 2004) takes the other route: it adds spherical types – points, circles, polygons, boxes – and indexes them with GiST over bounding volumes, so that regions can be stored and queried as data, not only as query arguments.

The scale this has to work at is set by archives already in operation. The ESA Gaia Archive (Salgado et al. 2017, 2020) serves a five-parameter astrometric catalogue of order 10^9 sources through an extended TAP interface, and its published description puts cross-match – matching a list of positions a user has uploaded against the catalogue – among the server-side capabilities implemented beside the query interface itself, under the principle of moving the code close to the data rather than the data to the user. Cross-match is therefore not a synthetic benchmark operation: it is a service an archive of this kind is expected to provide, and Sect. 6.6 treats it as a first-class result rather than an afterthought.

Neither idea is new to astronomy. The Hierarchical Triangular Mesh (Kunszt et al. 2001) had already keyed positions by a subdivided polyhedron, and the zones algorithm (Gray et al. 2006) showed how far a purely relational arrangement – declination strips and a B-tree – can be pushed for cross-matching. What has changed since is the size of the catalogues and the fact that most queries now arrive through a generated layer (e.g. Demleitner et al. 2014) rather than being written by hand, so the index has to serve a translator’s output rather than a tuned query.

The two designs trade off against each other. A linear key in a B-tree is compact, cheap to probe and orders the heap, but a region has to be turned into a set of key ranges before the index can be used, and a set of ranges is an approximation whose quality depends on how finely the region is cut. A tree of bounding volumes has no such approximation step and handles extended objects naturally, but its keys are larger, its nodes overlap, and it imposes no useful order on the table.

This paper presents `skycell`, which keeps the integer key and asks a cost question at the point where the classical approach fixes a constant: how finely should a region be cut? The answer depends on two quantities the database already knows – how many sources sit in that part of the sky, and how much an extra index range costs – and it can be computed rather than configured.

Because the interesting claims here are quantitative, the measurements are built to survive scrutiny rather than to flatter the method: every query is answered by all three methods back to back in a randomized order, repeated and analysed paired; everything is measured both warm and with the page

Table 1: Notation.

Symbol	Meaning
k	HEALPix order, $0 \leq k \leq 29$
s_k	cell size at order k , $s_0/2^k$ rad
A_k	cell area, $4\pi/(12 \cdot 4^k)$ sr
R	query region (cone or convex polygon)
A_{region}	area of R , sr
r	cone radius, rad
ρ	local source density, rows sr $^{-1}$
c_{range}	price of one index range, in rows
$C(s)$	cost of a covering of cell size s , rows
n_{ranges}	key ranges in the covering
A_{cover}	area covered by the covering, sr
$\text{maxpixrad}(k)$	centre-to-boundary bound, rad

cache dropped; and everything is repeated on a second corpus whose density structure comes from the real Gaia DR3 sky rather than from the authors. Section 2 develops the method, Sect. 3 its integration with a query planner, Sect. 4 what is proven and what is only tested, Sect. 5 the protocol, Sect. 6 the results, and Sect. 7 a TAP service and the measured crossover. Section 8 states plainly what these measurements do not establish.

2 Method

Table 1 collects the notation.

2.1 An equal-area integer key

A position is keyed by the index of the HEALPix cell containing it at order 29 in the nested scheme (Górski et al. 2005), computed by the standard `ang2pix_nest` arithmetic and stored as a signed 64-bit integer. At order 29 there are $12 \cdot 4^{29} \approx 1.4 \times 10^{19}$ cells of about 0.4 mas, so the key needs 63 bits and a PostgreSQL `bigint` holds it without reinterpretation.

The encoding is the plain nested index at a fixed order, not a multi-order key: the order is implicit, so a key is directly comparable with another key and a B-tree orders them along the HEALPix curve. Three properties follow, and only the first is shared with the cube quad-tree of Q3C.

Cells are *nested*: the order-29 descendants of a cell p at order k are exactly the contiguous interval

$$[p \cdot 4^{29-k}, (p+1) \cdot 4^{29-k} - 1], \quad (1)$$

so a region covered by cells of any orders becomes a union of integer intervals and a B-tree can answer it with range scans. The NUNIQ encoding of the MOC standard (Fernique et al. 2014), used here for stored regions (Sect. 2.5), packs the order into the key as $4 \cdot 4^k + p$; the two are related by Eq. (1), and `skycell` converts between them rather than storing two representations.

Cells are *equal area*. Every cell at a given order subtends the same solid angle, so a count of keys in

an interval divided by the interval’s length in cells is proportional to a source density per unit solid angle. The histogram that ANALYZE keeps for the key column is therefore a map of how crowded each part of the sky is (Sect. 2.3), without any extra statistics machinery.

Cells are *standard*. The same numbering underlies Gaia’s `source_id` (Gaia Collaboration 2016) and the IVOA’s Multi-Order Coverage maps (Fernique et al. 2014), so keys, coverings and stored footprints are interoperable rather than private to one extension.

2.2 Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer such false positives and more ranges. With cells of size s covering a region of characteristic radius r and a local source density ρ ,

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}). \quad (2)$$

Both terms are in rows: the second is the expected number of false positives, and c_{range} converts one additional index range into the number of false-positive rows it is worth avoiding. It is therefore a property of the installation, not of the sky: the ratio between the cost of opening one more B-tree range scan (a descent of the tree plus a planner path with its own selectivity estimate) and the cost of fetching and discarding one heap row. Section 6.8 measures it.

A covering follows the region’s boundary, so for $s \ll r$ the number of ranges grows as r/s – the curve enters and leaves the region once per boundary crossing, and the number of such clusters is the classical measure of a space-filling curve’s quality (Moon et al. 2001) – while the wasted area grows as the perimeter times s . Writing $A_{\text{cover}} \simeq \pi(r + \beta s)^2$ and $n_{\text{ranges}} \simeq \gamma r/s$ and minimising Eq. (2) gives

$$s_{\star} = \sqrt{\frac{\gamma c_{\text{range}}}{2\pi\beta\rho}}, \quad (3)$$

which does not depend on the size of the region: the density of the sky and the price of a range decide how finely it is worth cutting. For $s \gtrsim r$ the covering is a handful of cells and the balance differs, so rather than solve the branches the implementation evaluates Eq. (2) at all 30 orders – a few floating-point operations each – and takes the cheapest, which also lets a cap on the number of ranges and a guard on cell size simply remove candidates. The constants $\beta = 0.7$ and $\gamma = 2$ are geometric; Sect. 6.8 shows the choice of order is flat enough near its optimum that they do not need to be exact.

Figure 1 shows the two extremes on the same cone radius: at high Galactic latitude the covering is a single range of large cells covering 5.6 times the cone’s

area, because there is almost nothing to exclude; in the bulge the same cone is covered by 23 cells in 8 ranges covering 1.3 times its area, because there the rows removed are worth the extra ranges.

Two guards keep the model safe against its own inputs. A histogram is a coarse synopsis, and a dense cluster split by the space-filling curve into pieces smaller than one bucket can be underestimated; a partly covered cell is therefore never allowed to exceed a multiple (64 by default) of the region’s own area, which bounds the damage to the density-proportional behaviour a fixed-depth covering would have had. Gaps between ranges are merged when the rows they contain cost less than the range they save.

2.3 The density estimate

PostgreSQL’s ANALYZE stores, for the indexed expression, an equi-depth histogram: a sorted list of $B + 1$ bounds $h_0 < h_1 < \dots < h_B$ in key space such that each bucket $[h_i, h_{i+1})$ holds about N/B of the table’s N rows. Because the key is an equal-area cell index, the rows in a key interval $[a, b]$ are estimated by linear interpolation within the buckets it spans,

$$\hat{n}(a, b) = \frac{N}{B} \sum_i \frac{|[a, b] \cap [h_i, h_{i+1})|}{h_{i+1} - h_i}, \quad (4)$$

and the corresponding density in rows per steradian is

$$\hat{\rho}(a, b) = \frac{\hat{n}(a, b)}{(b - a + 1) A_{29}}. \quad (5)$$

The estimate used by Eq. (3) is $\hat{\rho}$ evaluated over the order- m cell containing the region’s centre, with m chosen so that $s_m \approx 4r$: large enough to contain enough rows to be a density, small enough to be local.

Three things are worth stating plainly. The quantity is a density per unit solid angle, not a selectivity: the planner’s own selectivity estimate for the rewritten range conditions is computed separately by PostgreSQL from the same histogram. The interpolation in Eq. (4) assumes rows are spread uniformly inside a bucket, which is exactly what fails for a cluster smaller than a bucket – hence the area guard of Sect. 2.2. And the estimate can only be as local as a bucket: with $B = 1000$ and $N = 10^7$ a bucket holds 10^4 rows, so below that scale Eq. (5) returns the average over a region containing 10^4 rows, which is the correct answer to a different question. Section 6.9 measures how close $\hat{\rho}$ lands to the truth, and at which scales the truth itself is defined.

2.4 Classifying cells: what is bounded and what is not

A cell must be classified against the region as inside, outside or straddling its boundary. The three outcomes are not symmetric. A cell called *outside* is dropped from the covering, so if that is ever wrong

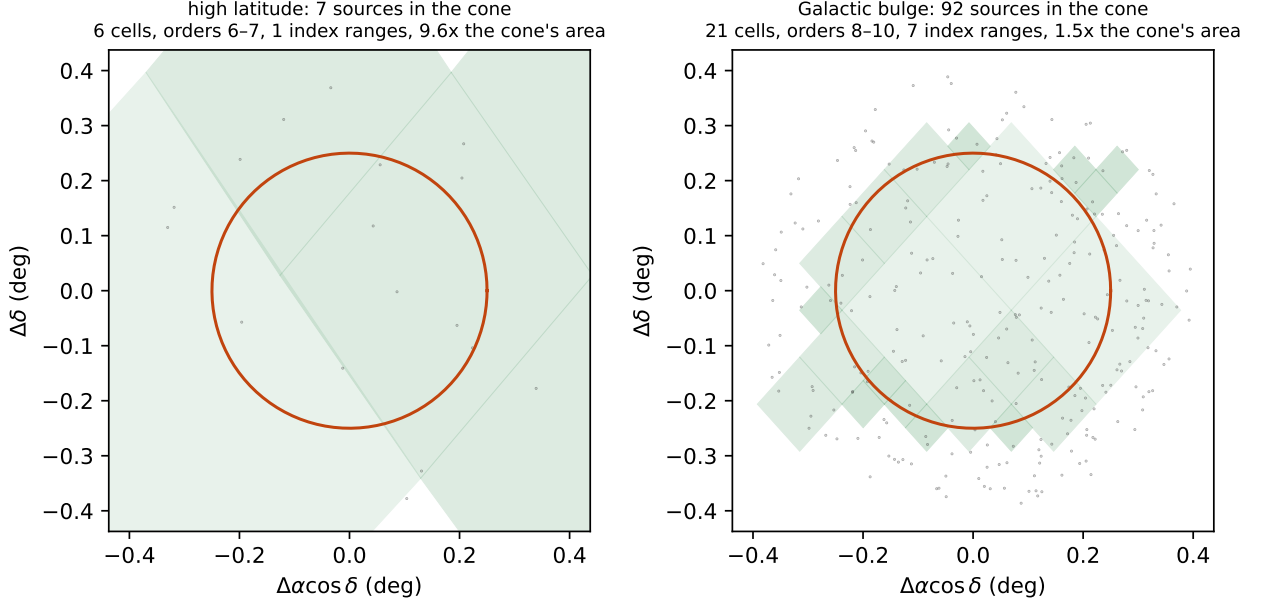


Figure 1: The same 0.25° cone covered in two places on the same catalogue. *Left*: at high Galactic latitude the cost model buys one index range of large cells, covering 5.6 times the cone’s area, because the 11 sources in the cone leave almost nothing to exclude. *Right*: in the Galactic bulge the same cone holds 1183 sources, and the model cuts two orders deeper – 23 cells in 8 ranges, 1.3 times the cone’s area – because there the false positives cost more than the ranges save. Grey dots are catalogue sources, the heavy circle the query cone, and the shaded quadrilaterals the covering cells, drawn from their own corners.

the query silently loses rows. A cell called *inside* is taken whole, and a cell called *straddling* is refined or taken whole; both produce a superset of the region, and the exact predicate that the rewrite conjoins to the index conditions (Sect. 3) removes whatever the covering over-included. Only the outside decision therefore needs a bound rather than an estimate.

The bound used is

$$R_{\text{cell}}(k, p) = \min(\text{maxpixrad}(k), \max_i \angle(\mathbf{c}_{k,p}, \mathbf{v}_i)), \quad (6)$$

where $\mathbf{c}_{k,p}$ is the cell’s centre and $\mathbf{v}_i$ its four corners. The first term is the analytic worst case over the sphere used by the reference HEALPix implementation; the second is the cell’s own extent, which is smaller for most cells. A cell is outside when $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ for a cone of centre $\mathbf{q}$ and radius r , and correspondingly for each of a polygon’s bounding half-spaces. The margin $\epsilon = 2 \times 10^{-13}$ rad (40 pas) covers the accumulated floating-point error of the handful of dot products, cross products and inverse trigonometric calls involved, each carrying a few units in the last place; it is five orders of magnitude below the 0.4 mas resolution of an order-29 cell, so it costs nothing.

The inside test may be, and is, tighter: it uses the distances from the query centre to the cell’s own corners and edge midpoints directly.

A tempting alternative – deciding *outside* from the distance to the chords between corners, inflated by the edge’s departure from its chord at the mid-

point – is not a bound: where an edge crosses its chord plane near the midpoint that departure collapses towards zero while the true one does not, by a factor reaching 64 at order 18. The error it caused was $\sim 10^{-8}$ rad, below a leaf cell and invisible to every test we had, which is why the covering uses Eq. (6) instead, at a cost of 2.5% more index ranges and no measurable time.

2.5 Regions as stored objects

A footprint is covered by a small number of cells written in the NUNIQ encoding of the MOC standard and stored one per row in an ordinary B-tree. The footprints containing a given position are those whose cell is one of that position’s ancestors, a short list of equality lookups; the exact geometry is then applied to the candidates. Since a MOC’s cells are disjoint by construction, no duplicate elimination is needed. Refinement of these coverings stops when cells become negligible next to the region, which keeps the ancestor list short.

2.6 The query surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so that a translator emits the standard’s spelling rather than a dialect. The operator form `pos <@`

`circle(...)` is the one a planner can answer from an index; the function form returns the same rows.

Beside the geometry, archives need the astrometry that decides *where* a source is at the epoch being asked about. `skycell` implements the IVOA UDF registry’s names – `ivo_epoch_prop`, `ivo_epoch_prop_pos`, `ivo_apply_pm`, `ivo_healpix_index`, `ivo_healpix_center` – with the rigorous epoch propagation of the Hipparcos catalogue (ESA 1997), in which the star follows a straight line in space so that parallax, proper motion and radial velocity all evolve together. Frame conversions between ICRS, Galactic and ecliptic coordinates are provided for the same reason.

An index built on catalogue positions cannot know where a star has moved to, so a proper-motion-aware cone search is written as an indexable cone widened by the largest motion of interest, intersected with an exact test on the propagated position.

Table 2 sets the three side by side: what ADQL says, what a translator must emit for the pgSphere-plus-Q3C pair, and what it emits for `skycell`. The right-hand column is mostly the identity, which is the point of adopting the standard’s spellings; the middle column is where the work goes, and three of its rows have no direct form at all. The astrometry functions are omitted from the table: all are exercised by round-trip and published-value tests (epoch propagation returns Barnard’s star to sub-microarcsecond over a 16-year round trip; the frame conversions put the Galactic centre at $l = b = 0$) and none is benchmarked, so no performance claim is made for them.

3 Implementation

The extension is written in C against the PostgreSQL 15+ server API and consists of a geometry layer with no database dependencies – HEALPix arithmetic, region predicates and the covering – and a layer that connects it to the planner.

The connection is a *planner support function* (`SupportRequestSimplify`), the mechanism PostGIS uses for `ST_DWithin`. Before path generation, a predicate such as `point('ICRS', ra, dec) <@ circle('ICRS',  $\alpha$ ,  $\delta$ ,  $r$ )` is replaced by a disjunction of range conditions on the key expression, conjoined with the exact test:

```
(cell BETWEEN lo1 AND hi1
OR cell BETWEEN lo2 AND hi2 OR ...)
AND skycell.in_region(point(ra, dec),
    circle(...), 0.29)
```

The key expression is not written by the user: the support function builds the candidate expression from the predicate’s position argument and keeps it only if the relation actually carries an index on it, so the same query text works whether positions are stored as two columns or as a `skypos` column. Because the index conditions are then ordinary range

predicates, the planner estimates their selectivity from the same histogram, and the last argument of the exact test carries the fraction of the covering that the region occupies, so that the product of the two estimates is close to the truth.

Two caches matter for planning cost. The density model and the identity of the matching index are cached per backend and dropped whenever statistics, the relation or the relation cache change; coverings are memoised by region and statistics source, since a service plans the same cone many times. Measured on a TAP corpus, the first cache is worth 45–70 μ s per plan and the second up to 50 μ s for larger regions.

For joins, where the region is not a constant, the rewrite emits a fixed number of range slots computed per outer row – the structure `q3c_join` uses – while a set-returning function `skycell_cone_ranges()` offers the alternative of a variable number of ranges through a `LATERAL` join. The two differ in plan shape rather than geometry, and Sect. 6.5 shows the difference is large.

Ingestion and maintenance. The index is an ordinary B-tree on an expression, so its maintenance behaviour is PostgreSQL’s: an insert adds one 8-byte key, updates that do not move a source do not touch it, and there is no rebalancing peculiar to the method. Two operational points differ from a geometry index. Bulk ingestion benefits from loading in key order, since that is also the heap order the queries exploit, and the cost of *not* doing so is the usual loss of locality rather than a degraded index. And the covering depends on the histogram, so statistics matter more than for a GiST index: a relation whose density structure has changed substantially since the last `ANALYZE` will be covered for the sky it used to have. The guard of Sect. 2.2 bounds the damage, and `ANALYZE` on the 10-million-row corpora here takes 0.5–1.4 s.

4 What is proven, and what is tested

4.1 A conservative-covering statement

Proposition. Let R be a cone or convex polygon and let $\mathcal{C}$ be the covering computed for it. If (i) every point of a HEALPix cell at order k lies within $\text{maxpixrad}(k)$ of that cell’s centre, and (ii) every angle computed by the classification carries an absolute error below ϵ , then every order-29 cell that intersects R lies in one of the ranges of $\mathcal{C}$.

Proof sketch. The covering is produced by a descent that starts from cells whose union is the whole sphere and, at each step, either keeps a cell, drops

Table 2: What a translator has to emit. For each ADQL construct: the pgSphere-plus-Q3C form an archive uses today, and the `skycell` form. Q3C supplies the fast cone and cross-match paths over scalar `ra/dec` columns but has no region type; pgSphere supplies the region types but is slower at cones and cross-matches (Sects. 6.1, 6.6), so a translator targeting the pair must choose between them per query shape, and the choice is not always available. `skycell` uses ADQL’s own spellings, so the translation is the identity except where an indexable form is wanted.

ADQL	pgSphere + Q3C	skycell
<code>POINT('ICRS', ra, dec)</code>	<code>spoint(radians(ra), radians(dec))</code>	<code>point('ICRS', ra, dec)</code>
<code>CIRCLE('ICRS', a, d, r)</code>	<code>scircle(spoint(...), radians(r))</code>	<code>circle('ICRS', a, d, r)</code>
<code>BOX('ICRS', a, d, w, h)</code>	<code>sbox</code> takes two corners, not a centre and extent: the translator converts, and the result is wrong at the poles, where a box is not a rectangle in (α, δ)	<code>box('ICRS', a, d, w, h)</code>
<code>POLYGON('ICRS', a₁, d₁, ...)</code>	<code>spoly('{(...), (...)}')</code>	<code>polygon('ICRS', a₁, d₁, ...)</code>
<code>CONTAINS(POINT, CIRCLE) = 1</code>	<code>q3c_radial_query(ra, dec, a, d, r)</code> or <code>pos <@ scircle(...)</code> – Q3C for speed, pgSphere if the position is stored as a type	<code>point(...) <@ circle(...)</code>
<code>CONTAINS(POINT, POLYGON) = 1</code>	<code>q3c_poly_query(ra, dec, array)</code> or <code>pos <@ spoly(...)</code>	<code>point(...) <@ polygon(...)</code>
<code>CONTAINS(POINT, BOX) = 1</code>	as <code>BOX</code> above	<code>point(...) <@ box(...)</code>
<code>INTERSECTS(r₁, r₂)</code>	<code>r₁ && r₂</code> (pgSphere only; Q3C has no region type)	<code>intersects(r₁, r₂)</code>
<code>DISTANCE(p₁, p₂)</code>	<code>q3c_dist(a₁, d₁, a₂, d₂)</code> or <code>degrees(p₁ <-> p₂)</code>	<code>distance(p₁, p₂)</code>
<code>AREA(region)</code>	<code>area(spoly)</code> returns steradians, so the translator scales by $(180/\pi)^2$; no Q3C equivalent	<code>area(region)</code>
<code>COORD1/COORD2(p)</code>	<code>degrees(long(p)), degrees(lat(p))</code>	<code>coord1(p), coord2(p)</code>
<code>CENTROID(region)</code>	no direct equivalent for a polygon	<code>centroid(region)</code>
cross-match (CONTAINS against an up-loaded table)	<code>q3c_join(a₁, d₁, a₂, d₂, r)</code> ; the pgSphere form is 2.4 times slower (Sect. 6.6)	<code>skycell_cone(...)</code> , or <code>skycell_cone_ranges()</code> in a LATERAL
<code>ivo_epoch_prop_pos(...)</code>	no equivalent in either	same name

it, or replaces it by its four children, whose union is the parent. The union of the kept cells is therefore reduced only where a cell is dropped, which happens only when it is classified outside. Taking $R_{\text{cell}} = \text{maxpixrad}(k)$ in Eq. (6), every point $\mathbf{x}$ of cell (k, p) satisfies $\angle(\mathbf{x}, \mathbf{c}_{k,p}) \leq R_{\text{cell}}$ by (i); hence if $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$, then by the triangle inequality on the sphere and (ii) every point of the cell is farther than r from $\mathbf{q}$, so the cell does not intersect the cone. The polygon case is the same argument applied to each bounding half-space. Truncation by the range cap, the area guard or the step budget only stops refinement, which keeps a cell whole, so it cannot remove a cell either. A cell containing a point of R intersects R and is therefore never dropped. $\square$

The proposition is conditional, and both conditions deserve their status. Condition (i) is a property of HEALPix, not of this work: the reference implementation’s `maxpixrad` is analytic, and we verify it numerically over 1.6×10^6 sampled boundary points at every order, the largest observed ratio of centre-to-point distance to `maxpixrad` being 0.9999. The tighter corner term in Eq. (6) additionally assumes that the extremum of the distance from the centre over a cell is attained at a corner, which the same scan supports (largest observed ratio 0.96) but which we have not proven; a deployment that wants the proposition to rest on (i) alone can set `skycell.exact_cells` to off, at the cost quantified

in Sect. 2.4. Condition (ii) is an assumption about floating-point arithmetic rather than a theorem: ϵ is chosen five orders of magnitude above the plausible accumulated error and five below the leaf cell.

4.2 Brute force

Because a covering that misses a cell silently loses rows, the implementation is checked by exhaustion at three levels.

The HEALPix layer is checked against known cell centres, round trips between positions and cells at all 30 orders, the nesting property of Eq. (1), equal area ($\chi^2/\nu = 1.14$ over 768 base cells), and the bounds of Sect. 2.4, over 1.6×10^6 sampled points including the poles, the boundary between the polar and equatorial zones, and the base-pixel edges.

The covering is checked against a 2-million-point clustered catalogue for cones from $1''$ to 120° and for convex polygons; against 30 000 adversarial cones – at the poles, across the zone boundary, at $\alpha = 0$ and on face edges, with radii from 0.02 mas to 10° – with 200 points sampled inside each, half of them within a part per billion of the rim; and against 7266 adversarial polygons at the same awkward places, with 866 537 points drawn inside them, again half of them a hair inside an edge or a vertex. All assert zero false negatives, in both the seeded and the unseeded covering path. This is the test that caught the error

described in Sect. 2.4.

At the SQL level, two regression suites require that every indexed answer equals the answer from a sequential scan with the exact predicate: 240 cones in two configurations, 60 polygons, both join forms and the MOC footprint lookup in the first; the ADQL surface, the operator and function spellings over 120 random regions, the presence of index plans (and their absence when the region is not constant), epoch-propagation round trips and frame conversions in the second.

5 Benchmark

5.1 Two corpora

The first corpus is 10 million synthetic sources with a density contrast resembling a real optical survey: 35% uniform on the sphere, 45% in a Galactic disk (longitude uniform, latitude exponential with a 3° scale), 12% in a bulge (Gaussian, 7°), and 8% in 200 clusters with dispersions from 0.02° to 0.5° . Positions are drawn deterministically from a seed. We call it the *designed* corpus, because its density structure was chosen by the authors of the method under test, which is exactly the objection such a corpus invites.

The second corpus removes that objection as far as we can. We queried the ESA Gaia archive for the number of Gaia DR3 sources in every HEALPix cell at order 9 (3 145 728 cells of 0.11° ; the counts sum to 1 811 709 771, the published DR3 total), and drew 10 million positions from that density field: each cell receives a number of sources proportional to its Gaia count, placed uniformly inside it by choosing a random order-29 descendant, which is exact because the descendants are equal area. The result has the real sky’s crowding – the disk, the bulge, the Magellanic Clouds, the scanning-law texture – down to the 0.11° scale of the map, below which it is locally Poisson. It is not a Gaia catalogue: no real positions are used, and structure finer than a map cell is absent by construction. What it tests is whether the cost model, which reads density at scales of a few times the query radius, behaves the same on a density field the authors did not invent. We call it the *Gaia* corpus.

5.2 Targets and layout

Q3C 2.0.5, pgSphere 1.5.2 and `skycell` each hold the same logical rows in their own natural layout, all three physically ordered along a space-filling curve so that heap locality is held equal (Table 3). Queries are the documented interface of each extension: `q3c_radial_query`, `spoint <@ scircle` and `skycell_cone`. Row counts are compared on every query.

5.3 Protocol

The measurement is organised as *trials*. A trial is one query centre and radius; within a trial all three methods answer the same query back to back, in an order drawn at random for that trial, and the position each method drew is recorded so that an order effect can be measured rather than assumed away. Trials are repeated five times over the whole query set, and the query set is traversed in a random order each repetition. The analysis is paired: for each query we take the median over repetitions per method and form the ratio between methods for that same query, so that the query’s own difficulty cancels; the reported interval is a percentile bootstrap (4000 resamples) over queries. A ratio is called a difference only when its interval excludes 1.

Everything is measured in two cache regimes. *Warm*: an untimed pass over the whole query set precedes the measurement, so no method is measured on a colder cache than another. *Cold*: the server is restarted, so its 2 GB of shared buffers is empty, and the virtual machine’s page cache is dropped, so the data must come from storage; only the first repetition is used, since a second would no longer be cold.

Separately, a randomized pass under `EXPLAIN (ANALYZE, BUFFERS)` records planning time, execution time, buffer pages and the planner’s estimate against the actual count, so that planning and execution can be reported apart from each other.

Cone radii span $1''$ to 3° with 16–400 queries each, half of the centres drawn from catalogue positions (hence density-weighted) and half uniform on the sky. Cross-matches join a 200 000-row probe table – half catalogue sources displaced by $\sim 0.3''$, half uniform – to the catalogue. Footprint queries ask which of 20 000 stored circles contain each of the 200 000 probes.

5.4 Reproducibility

Table 3 records the configuration in the detail needed to repeat or contest the measurement. Everything runs on one machine, with one core doing one query.

5.5 What this protocol still does not control

The corpora are synthetic, one designed by the authors and the other derived from a real density map but not from real positions. One machine, one storage device, one PostgreSQL configuration. Absolute times are properties of that machine; only the comparisons are meant to travel. The cold regime drops the page cache but cannot empty the device’s own cache. And the three extensions are compared as configured here: a different Q3C query formulation or pgSphere index configuration could move their

Table 3: The measured configuration.

Item	Value
Host	Apple silicon laptop, 11 cores, Docker Desktop Linux VM, 8 GB
Storage	NVMe SSD, through the VM’s file system
PostgreSQL	18.6; <code>shared_buffers</code> 2 GB, JIT off, <code>max_parallel_workers_per_gather</code> 0
Extensions	Q3C 2.0.5, pgSphere 1.5.2, skycell 0.4
<code>range_cost</code>	derived per relation (Eq. 7); 116 for the catalogues here
Rows	10^7 per corpus (Gaia corpus: 9999729); the Gaia corpus also at 5×10^7 (50027391)
Q3C	<code>cat_q3c(id, ra, dec, mag)</code> , B-tree on <code>q3c_ang2ipix(ra, dec)</code> , heap ordered by the same
pgSphere	<code>cat_sphere(id, ra, dec, mag, pospoint)</code> , GiST on <code>pos</code> , heap ordered by the HEALPix key
skycell	<code>cat_cell(id, ra, dec, mag, cellint8)</code> , B-tree on <code>cell</code> , statistics target 1000, heap ordered by <code>cell</code>
Maintenance	<code>VACUUM (FREEZE)</code> and <code>ANALYZE</code> on all three before measuring
Repetitions	5 (warm), 1 (cold); 1456 queries per repetition
Uncertainty	percentile bootstrap, 4000 resamples, over queries

numbers, and Sect. 6.1 reports one such effect that dominates Q3C’s small-cone results.

6 Results

All three methods return identical row counts on every query of the grid. In the cross-matches pgSphere returns one or two more pairs out of $\sim 100\,000$, consistent with its 10^{-9} rad comparison tolerance counting sources exactly on the rim.

6.1 Cone searches

Table 4 and Fig. 2a are the primary result: paired ratios with intervals, on both corpora, warm and cold.

Warm, **skycell** is faster than pgSphere by 6–11% for radii from $1''$ to $1'$ and by 31–37% at 3° , on both corpora. Between $6'$ and 1° the two are, on the designed corpus, indistinguishable; on the Gaia corpus skycell is 3% *slower* at $6'$ (interval [1.00, 1.08]) and 8–15% faster at $30'$ and 1° . This is a smaller and more ragged advantage than a phase-ordered measurement suggests (Sect. 6.3), and it is the number we stand behind.

Cold, the picture is different and more favourable: skycell is faster at every radius on both corpora, by 5% at $1'$ and by a factor 2–2.5 at $30'$ and beyond. The reason is visible in the buffer counts of Table 5: at 1° skycell touches 55 pages against pgSphere’s 154.

When those pages are in memory the difference is a few microseconds of memory traffic; when they must be read it is the whole cost of the query. An index that wastes less area is worth most exactly when the data does not fit in cache, which is the regime a large archive operates in.

Against Q3C, **skycell** is faster by a factor 4–50 depending on radius and cache state. The gap at small radii has a specific and reportable cause: Q3C 2.0.5 expands every `q3c_radial_query` into 100 key ranges regardless of radius, which costs about 300 buffer pages and 1.2 ms of planning before any row is read (Table 5). This is a property of that version’s query path rather than of the cube quad-tree; Q3C’s cross-match path, which uses four ranges per probe, does not pay it, and Sect. 6.5 shows it performing accordingly.

The order effect the randomization exists to control is small but real: averaged over all radii on the designed corpus, going first rather than last costs pgSphere 19% and skycell –5% warm, while Q3C is flat. Since methods are assigned to positions at random, this inflates the variance of the ratios rather than biasing them.

6.2 Does the advantage grow with the catalogue?

Ten million rows is 0.55% of Gaia DR3, and the argument that an index matters most when it does not fit in memory is an argument about scale. We therefore repeated the whole protocol on a 50-million-row Gaia-density corpus, built identically. The point of that size is specific: pgSphere’s GiST index is then 3442 MB against a 2 GB `shared_buffers`, so it no longer fits, while **skycell**’s B-tree is 1072 MB and still does.

The answer (Table 6) divides by region size. For $30'$ and larger the advantage grows markedly with the catalogue: at 1° the warm ratio improves from 0.85 to 0.60 and the cold ratio from 0.53 to 0.31, and the buffer counts show why – 152 pages against pgSphere’s 514, where at 10^7 rows it was 55 against 154. For cones of an arcminute and below nothing improves: $1''$ stays at 0.89, and $1'$ moves from 0.93 to 1.02, no longer distinguishable from pgSphere. The reason is in the same table. At those radii both methods touch five to seven pages whatever the table size, so there is no scan left to shorten, and what remains is **skycell**’s planning – 0.016 to 0.020 ms against pgSphere’s flat 0.009 – which grows with source density because a denser sky makes Eq. (3) cut finer.

The rule that emerges is about pages rather than rows: **skycell** wins wherever a query has enough pages to save, and the larger the catalogue the more that is worth. An arcsecond cone on any catalogue is not the workload this index improves; a degree-scale selection, a polygon or a cross-match on a large one is.

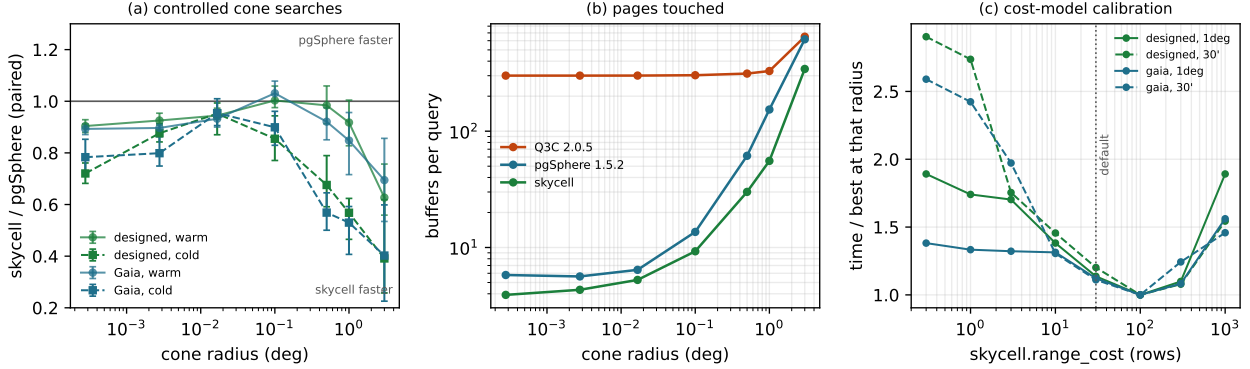


Figure 2: (a) The controlled comparison: the paired ratio of `skycell` to `pgSphere` against cone radius, with 95% bootstrap intervals, for both corpora and both cache regimes. Below the line `skycell` is faster. The warm advantage disappears around 0.1° and returns at degree scales; with caches dropped it holds at every radius. (b) Buffer pages touched per query, which is what the cold curves are made of: at 1° `skycell` reads a third of `pgSphere`'s pages, and Q3C pays about 300 pages at any radius because version 2.0.5 always expands a radial query into 100 key ranges. (c) The cost model's one free parameter, swept over four decades and normalised to the best setting at each radius. The curve is flat-bottomed over a decade either side of its minimum, which is why deriving the parameter from the relation (Sect. 6.8) costs nothing in speed and why the model's choice of cell order is within a few percent of the best available.

Table 4: Cone searches: paired ratio of `skycell` to each rival, with a 95% percentile-bootstrap interval over queries. A value below 1 means `skycell` is faster. **Bold** marks an interval that excludes 1 with `skycell` faster; *italic* marks one that excludes 1 with `skycell` slower; plain type means the two are not distinguishable. n is the number of distinct queries at that radius; each is answered five times per method (warm) or once (cold), all three methods in a randomized order within each trial.

radius	n	designed corpus		Gaia corpus					
		warm	cold	warm	cold				
<i>skycell</i> / <i>pgSphere</i>									
1''	400	0.90	[0.88, 0.93]	0.72	[0.68, 0.76]	0.89	[0.87, 0.92]	0.78	[0.72, 0.85]
10''	400	0.93	[0.91, 0.96]	0.88	[0.84, 0.92]	0.90	[0.88, 0.91]	0.80	[0.75, 0.86]
1'	300	0.94	[0.93, 0.97]	0.95	[0.87, 1.00]	0.93	[0.90, 0.96]	0.95	[0.90, 1.01]
6'	200	1.00	[0.98, 1.06]	0.85	[0.77, 0.94]	<i>1.03</i>	[1.00, 1.08]	0.90	[0.83, 0.96]
30'	100	0.98	[0.91, 1.06]	0.68	[0.59, 0.79]	0.92	[0.85, 0.98]	0.57	[0.50, 0.65]
1°	40	0.92	[0.83, 1.00]	0.57	[0.47, 0.62]	0.85	[0.72, 0.96]	0.53	[0.41, 0.59]
3°	16	0.63	[0.56, 0.76]	0.39	[0.26, 0.62]	0.69	[0.53, 0.86]	0.40	[0.22, 0.58]
<i>skycell</i> / <i>Q3C</i>									
1''	400		0.02		0.14		0.02		0.14
1'	300		0.02		0.18		0.02		0.19
30'	100		0.06		0.42		0.06		0.41
1°	40		0.09		0.59		0.13		0.51
3°	16		0.27		0.75		0.37		0.66

Table 5: Planning and execution separated, with buffer pages and the error of the planner’s row estimate (median of $\max(\text{est}/\text{act}, \text{act}/\text{est})$), from a randomized EXPLAIN (ANALYZE, BUFFERS) pass on the designed corpus.

radius	method	plan (ms)	exec (ms)	buffers	est. err
1''	Q3C	1.230	0.213	300	1.00
	pgSphere	0.008	0.017	5	1.00
	skycell	0.015	0.007	4	1.00
30'	Q3C	1.242	0.291	313	1.92
	pgSphere	0.006	0.106	61	2.92
	skycell	0.034	0.075	30	1.16
1°	Q3C	1.224	0.402	329	1.89
	pgSphere	0.006	0.324	154	2.78
	skycell	0.062	0.145	55	1.22

6.3 How much the protocol matters

Measuring one method at a time instead – each method’s whole grid, then the next – gives *skycell/pgSphere* ratios of 0.69 to 0.87 below 30' where the controlled protocol finds 0.89 to 1.03. The phase-ordered arrangement flatters whichever method is measured later, and that was *skycell*. We note it because the difference is larger than most of the effects this paper measures, and it is the reason every comparison here is paired inside a trial.

6.4 Index size, build time, and what they cost at Gaia’s scale

The B-tree of 64-bit keys is 214 MB at 10^7 rows and builds in 1.4–2.3 s; pgSphere’s GiST is 685 MB and builds in 42–44 s (Table 7). For an archive this is an operational number: it is what an ingest, a restore or a schema migration has to pay, and it is the one result here that is not sensitive to protocol.

Index size is the one result in this paper that is insensitive to protocol, and it is also the one that decides deployments, so it is worth carrying to the scale an archive actually runs at. The cost per row is flat: between 10^7 and 5×10^7 rows it changes by 0.5% for pgSphere and not at all for the two B-trees (Table 8), which is what the structures predict – an 8-byte key with its tuple header in a B-tree, a bounding volume per entry in a GiST tree.

Extrapolating to the 1.81×10^9 sources of Gaia DR3 gives the right-hand columns of Table 8. The comparison that matters there is not one index against another but one *deployment* against another. pgSphere answers cone searches and stores regions; *q3c_join* is what PostgreSQL archives use for cross-matching, and Sect. 6.6 measures it 2.4 times faster than pgSphere at that. An archive that wants both therefore carries both indexes today. *skycell* answers both from one, and that one is the smaller kind.

The memory column is the operational point. A positional index is only as fast as its residency: the

measurements of Sect. 6.1 differ by a factor two to three between the warm and the cold regime, and which regime a server is in is decided by whether the index fits beside the heap’s working set. At DR3 scale a 64 GiB server holds *skycell*’s 38 GiB index with room to spare; pgSphere’s 122 GiB does not fit in 128 GiB once the heap is accounted for, and the pair needed to cover both cone searches and cross-matching wants 256 GiB. That is a factor four in the memory the machine must have, which at these sizes is a procurement decision rather than a tuning one – and it is exactly the regime where our cold-cache measurements put *skycell* furthest ahead (0.31 to 0.53 of pgSphere’s time at 30' and beyond).

The comparison a constrained machine actually faces. Table 8 invites matching regimes – warm against warm – but that is not the choice in front of an archive with, say, 64 GiB. There, *skycell*’s index is resident and the pgSphere-plus-Q3C pair is not, so the honest comparison is *warm skycell against cold pgSphere*. On the 5×10^7 -row corpus, where we measured both regimes:

radius	skycell warm	pgSphere warm	pgSphere cold	warm skycell / cold pgSphere
1''	0.042	0.047	0.528	0.08
1'	0.053	0.052	0.601	0.09
6'	0.092	0.085	1.168	0.08
30'	0.430	0.568	5.759	0.07
1°	0.875	1.450	18.401	0.05
3°	7.144	11.63	79.180	0.09

All times in ms. Matched warm, the two are within 10% at small radii; with the residency the index sizes actually buy, *skycell* is one to two orders of magnitude ahead. The same holds against Q3C, whose cold times are 2.1–21 ms.

This is an upper bound on the advantage, and should be read as one. Our cold regime is a fully cold start – shared buffers emptied and the page cache dropped – whereas a server whose index merely does not fit thrashes at some hit rate between the two columns rather than missing every time. The true figure depends on how much of the index fits and on the locality of the query stream, neither of which we have measured at that scale. What the measurement does establish is the direction and that the effect is large: the difference between the two designs stops being a matter of percent and becomes a matter of factors precisely when the index outgrows the machine, which is the regime a catalogue of 10^9 sources puts a server in.

Four caveats. The extrapolation is linear and validated only over a factor five in row count, though the per-row cost is constant to 0.5% across it. These are positional indexes alone: a real archive’s table, its other indexes and its ancillary products dwarf them. A partitioned deployment changes the arithmetic, since only the partitions in play need be resident. And the two B-trees are the same size, so the

Table 6: The same protocol at two catalogue sizes on the Gaia density field: paired ratio of **skycell** to pgSphere with 95% bootstrap intervals, and the planning, execution and buffer medians at 5×10^7 rows that account for them. At that size pgSphere’s index (3442 MB) exceeds **shared.buffers** while **skycell**’s (1072 MB) does not. **Bold**: interval excludes 1, **skycell** faster; *italic*: excludes 1, **skycell** slower.

radius	warm		cold		plan (ms)		exec (ms)		buffers	
	10^7	5×10^7	10^7	5×10^7	pgS	sky	pgS	sky	pgS	sky
1''	0.89	0.89	0.78	0.71	0.009	0.016	0.025	0.014	6	5
10''	0.90	0.94	0.80	0.88	0.008	0.016	0.025	0.014	5	5
1'	0.93	1.02	0.95	0.95	0.009	0.020	0.033	0.023	7	6
6'	<i>1.03</i>	1.00	0.90	0.89	0.009	0.031	0.059	0.049	14	19
30'	0.92	0.72	0.57	0.49	0.009	0.075	0.550	0.332	141	74
1°	0.85	0.60	0.53	0.31	0.009	0.087	1.976	1.040	514	152
3°	0.69	0.58	0.40	0.23	0.014	0.143	10.686	5.801	1080	676

comparison is really one geometry index plus one integer index against one integer index – the saving comes from not needing the geometry index at all, which rests on **skycell** answering the region queries pgSphere is kept for, as Sects. 6.1 and 6.5 measure.

Table 7: Storage and build cost for the same 10 million rows (Gaia corpus; the designed corpus agrees to within 5% except for **ANALYZE**). Table sizes differ because each method stores its own position representation alongside the catalogue columns.

method	table (MB)	index (MB)	build (s)	ANALYZE (s)
Q3C	575	214	2.2	0.23
pgSphere	731	685	42.3	0.46
skycell	652	214	1.4	0.57

6.5 Polygons and stored footprints

Convex polygons of 0.05° – 2° take a median 0.281 ms on the Gaia corpus against pgSphere’s 0.767 ms and Q3C’s 1.864 ms, and 0.142 ms against 0.631 and 1.526 on the designed corpus: a factor 2.7–4.4 either way; pgSphere’s mean (4.2 ms) and 95th percentile (21 ms) are far above its median, which the per-query records trace to polygons whose bounding volume is a poor fit.

6.6 Cross-matching

Cross-matching a user’s uploaded target list against the catalogue is the operation the Gaia archive lists among its server-side capabilities (Salgado et al. 2017), and it is Q3C’s own speciality: **q3c_join** is what most PostgreSQL archives use for it. It therefore deserves the same protocol as the cone searches rather than a single end-to-end run, and getting that protocol right turned out to matter more here than anywhere else in this paper.

The probe table is split into eight blocks of 25 000 probes, and a block is the unit of pairing: within a block all four methods run back to back in an order drawn for that block. Cache residency has to be controlled explicitly. At 50 million rows the three catalogues and their indexes are about 14 GB against 2 GB of **shared.buffers**, so a block’s first touch is I/O bound – the same **skycell** block query takes 6.2 s cold and 0.32 s once its pages are resident. Whichever method runs first on a block therefore pays for the others, and that effect is larger than the difference between the methods: in a first version of this measurement, running first cost **skycell** a factor ten while leaving Q3C and pgSphere, whose own cost is several seconds, visibly unchanged. Each method is consequently warmed on its own query for its own block immediately before being timed,

Table 8: Index cost per row, measured at two scales, and what it implies for a catalogue the size of Gaia DR3 (1.81×10^9 sources). Build times scale linearly from the 5×10^7 measurement, which is optimistic for all three equally. The last column is the smallest conventional server memory that holds the index resident with room left for the heap’s hot pages.

index	B/row		at 1.81×10^9		
	10^7	5×10^7	GiB	build	GiB
skycell	22.5	22.5	38	14 min	64
Q3C	22.5	22.5	38	13 min	64
pgSphere	71.8	72.2	122	2.7 h	192
pgSphere + Q3C	—	—	160	2.9 h	256

Table 9: Cross-matching 200 000 probes against 50 million sources, steady state: total time and the paired ratio to each rival over 16 block-repetitions. **Bold**: interval excludes 1, the method in the row is faster; *italic*: interval excludes 1, slower. All methods return identical row counts.

method	total (s)		ratio at 1''	
	1''	10''	vs Q3C	vs pgS.
q3c_join	1.23	1.32	—	0.42 [0.36, 0.49]
pgSphere	2.95	3.22	<i>2.39</i> [2.04, 2.82]	—
skycell, slots	1.50	1.85	1.07 [0.89, 1.48]	0.44 [0.38, 0.55]
skycell, lateral	1.08	1.43	0.78 [0.62, 1.00]	0.30 [0.26, 0.42]

so that all four are measured in the same state and what is compared is the work each does.

Table 9 gives the result at 50 million rows. All four methods return identical row counts in every block.

q3c_join is the fastest of the two established methods by a factor 2.4, which is what its reputation predicts. skycell’s LATERAL form is level with it – 0.78 of its time at 1'' with an interval that reaches 1.00, and 0.92 at 10'', not distinguishable – and 2.5 to 3.3 times faster than pgSphere. The join form, which emits a fixed number of range slots per outer row in the manner of q3c_join, is level with Q3C at 1'' and 35% slower at 10'', where a fixed slot count fits the covering less well.

The claim this supports is the narrow one: on the operation Q3C is built for, skycell is Q3C’s equal rather than its better, while being two and a half to three times faster than pgSphere – and it reaches that position with the same index that answers the region queries of Sect. 6.1, which is the point of the design.

For stored footprints, the MOC-in-a-B-tree arrangement answers the same 200 474 point-in-region pairs in 0.76 s against pgSphere’s 0.97 s, with an index that is larger (9.5 MB against 1.1 MB) and slower to build (0.9 s against 0.03 s), since a region becomes several rows rather than one.

These two workloads are measured one method after another rather than in randomized trials, because each is a single query of seconds rather than a population of sub-millisecond queries; the ordering effect of Sect. 6.3 applies to them.

6.7 Estimates, plans and joins

A spatial predicate is rarely alone. We ran three shapes – a cone with a catalogue cut, the same joined to an observation table, and a three-way join – through each method, recording the estimate, the plan strategy and the time separately, because they are different claims.

The estimate advantage carries over: for queries returning rows, the median estimate error at 30'–1° is 1.15–1.88 for skycell against 1.63–2.33 for Q3C and 2.50–3.00 for pgSphere.

The plans, however, are the same. Across 135 queries at 1', 30' and 1°, all three methods chose the same strategy (nested loop over index scans) for every query; skycell’s better estimate did not change what the planner decided. Only at 3°, where a cone returns ~15 000 rows, did one query out of sixteen cross a threshold: skycell’s estimate led to a hash join where pgSphere’s led to a nested loop, and that query ran 1.29 times faster. Execution times in the join shapes are otherwise within noise of pgSphere at 1'–1°.

The honest summary is that better estimates are demonstrated, better plans are not, and better join times are not. The value of an accurate estimate is insurance against the plan the optimiser would otherwise choose in a query we did not write; we have not shown that insurance paying off on a workload we did write.

6.8 The cost model’s one parameter, and how to stop setting it

Equation (2) has one free parameter, c_{range} , and it need not be a parameter at all: it is a ratio of two costs PostgreSQL already models. One more index range is a B-tree descent, which `btcostestimate` charges as $\lceil \log_2 N \rceil \cdot 50 c_{\text{op}}$ for the comparisons plus $(H + 1) \cdot 50 c_{\text{op}}$ for the pages; one false-positive row costs its index entry, its heap tuple, the exact predicate, and the heap page amortised over the rows that share it:

$$c_{\text{range}} = \frac{(\lceil \log_2 N \rceil + H + 1) 50 c_{\text{op}}}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + c_{\text{page}}/n_{\text{pp}}}, \quad (7)$$

with n_{pp} = reltuples/repages the rows per heap page. Every input is already at hand – repages sits beside reltuples in the `pg_class` tuple the density model loads, and the cost factors are the planner’s own GUCs – so `skycell.range_cost` defaults to evaluating it.

The last term is what makes this per-relation rather than per-installation. On the catalogues, at 120 rows per page, Eq. (7) gives 116; on an ObsCore-shaped relation, whose wider rows pack 54 to a page so that each false positive costs more page traffic, it gives 93, asking for a finer covering; with the stock `random_page_cost` of 4.0 instead of this host’s 1.1 it gives 67.

What the derivation buys is not speed. Placed in the same randomized trials as the fixed constant it replaces, the two are indistinguishable at every radius on the 50-million-row corpus (ratios 0.93 to 1.08, every interval containing 1), because the cost curve is flat: sweeping c_{range} over four decades moves

the median time by less than 25% anywhere between 3 and 300, and at 1° every setting from 3 to 300 lies within 3%. What it buys is that the parameter stops being a number someone has to choose, adapts to row width and to the storage the operator has described to the planner, and cannot be left at a value tuned for a different relation.

The order the model picks is likewise close to optimal: measured with two orders interleaved per query, its choice costs about 3% against one order finer at 6'. The flat curve is again the reason, and it is why the cost model does not need to be right – it needs to land in the flat region, and it does.

6.9 The density estimate against the truth

The cost model's input is $\hat{\rho}$ from Eq. (5). Compared with the rows actually counted in the same footprint, it is unbiased where a density exists: the median ratio $\hat{\rho}/\rho_{\text{true}}$ is 1.02 for the footprints a 1° cone reads (22 548 rows in them), 1.02 at 30' (5452 rows), 0.89–0.91 at 6' (165), and 0.94 at 1' (39), with the 10th–90th percentile band widening from [0.92, 1.16] to [0.63, 1.34] as the footprint empties – which is shot noise in the reference, not error in the estimate. Below about an arcminute the comparison stops meaning anything, because the footprint the model reads holds half a row on average; that is also the regime in which Sect. 6.8 shows the answer does not matter, since the covering is one cell whatever density is assumed. An equi-depth histogram is a poor two-dimensional density estimator in principle (Sect. 2.3); these measurements say that in practice it is good enough at the scales where its answer changes a decision.

7 Inside a TAP service, and where the crossover is

7.1 The service-level A/B

A benchmark of index operations is not a benchmark of an archive. We ran the same build inside **egernia** (Delli Veneri & Salgado 2026), a TAP 1.1 server, against its published 500 096-row ObsCore corpus: one database holding both indexes, two otherwise identical service processes differing only in how they translate **CONTAINS**, and the service's own harness interleaving them rung by rung over six query classes at 1, 8 and 32 concurrent clients, three repetitions each.

Both targets pass the conformance gate (**taplint**, zero errors) and the cross-server agreement gate on all eleven portable query classes, and the 203 cone queries of the corpus return identical rows through both translations. The performance result is a tie: all 18 comparison cells fall inside the tie rule, with

Table 10: Crossover on an ObsCore-shaped relation, measured in randomized paired trials (30 centres, 3 repetitions, each method warmed on its own query first): total query time as the ratio **skycell/pgSphere**, with buffer pages at the largest size. Q05: 0.15° cone; Q06: 2° cone; Q07: 0.5° cone with calibration and time cuts; Q12: an empty 0.7'' cone. The Q05 row is regime-dependent and should not be read as a single number (text).

class	skycell / pgSphere			buffers at 10 ⁷	
	5 × 10 ⁹	2 × 10 ⁶	10 ⁷	pgSphere	skycell
Q05	1.84	1.81	2.16	282	152
Q06	0.92	0.87	0.67	6052	830
Q07	0.84	0.76	0.71	712	182
Q12	1.13	1.07	1.00	5	6

throughput ratios between 0.96 and 1.06, although at the database level **skycell** is 1.2–1.45 times *slower* per query on that corpus.

Both facts have the same cause. A cone over half a million rows returns tens to hundreds of rows and costs a fraction of a millisecond either way, against roughly 10 ms of CPU per request spent in the service's own Python; the database is 1–3% of a request, so a difference of 0.08 ms in it cannot move the throughput of the service.

7.2 Where the crossover is

That explanation makes a testable prediction, and an earlier version of this work asserted it rather than testing it. We built the same ObsCore-shaped relation – observations in groups of 128 products sharing a field centre, clustered by collection and field, which is what makes a GiST bitmap scan so effective on this data – at three sizes, and ran the same four cone classes against both indexes (Table 10).

The crossover is real, but it is not a single number. For the cone with catalogue cuts (Q07) and the degree-scale cone (Q06), **skycell** is ahead at every size and pulls further ahead as the relation grows, reading a seventh of pgSphere's buffer pages at Q06 and a quarter at Q07; the empty cone (Q12) is level. The small-cone class (Q05, 0.15°) we cannot pin down: paired inside a trial it is 2.16 times slower immediately after the build, and level (0.94) on the same relation once thoroughly warm. Those are different cache regimes, and on this host we could not make that one class reproducible to better than a factor of two. The deficit is at least localised – it is in execution (0.324 ms against 0.161), not planning (0.034 against 0.009), and not the exact predicate, whose removal changes the time by about 1% here against 63% on the catalogue.

7.3 When the index no longer fits in memory

Section 6.4 argues from index size alone that a host must reach a point at which pgSphere’s index no longer stays in memory while `skycell`’s still does. That is an argument about arithmetic, so we tested it. A container limited to 3 GiB, with 768 MB of shared buffers, holds an ObsCore-shaped relation of 20 485 632 rows whose heap is 22 GB. The GiST index over `spoint` is 1162 MB (59.5 B/row, $1.51\times$ the buffer pool); the `skycell` expression index is 439 MB (22.5 B/row, $0.57\times$).

Making the cache matter at all required a query stream that sweeps the sky. Twenty thousand cones of 0.15° cover 3.4% of it and read essentially nothing from disk at either index size: what has to fit is not the index, but the part of it the stream touches. We therefore used 3500 cones of 2° drawn uniformly, which cover the sphere about once.

Each index was measured in its own phase, with the server restarted between phases so that the buffer pool starts empty and the index not under test occupies none of it – checked after every phase, where it holds 0 bytes. Each phase warms itself with an untimed sweep and then measures a second sweep in a fixed random order; both orderings of the two phases were run (Table 11). `skycell` answers the stream 4.2–4.8 times faster, and the mechanism is visible in the index traffic: it serves 53% of its index accesses from the pool against pgSphere’s 9%. The buffer counts reproduce to within one page across the two orderings, so the spread in the medians is host drift rather than an ordering effect.

Two qualifications matter. First, this is not the clean threshold the size arithmetic suggests. The 22 GB heap takes 705–734 MB of the 768 MB pool in every phase, so *neither* index is resident; what is being measured is a smaller index winning a contested pool, not one index fitting while the other does not. Second, the effect is easy to measure away. Running both translations in a single pass over the same centres – the obvious design, and the one we tried first – puts the gap at $1.6\times$ rather than 4.2, because the two answer the same cone from the same heap pages and each leaves them warm for the other.

8 Discussion

These measurements support a bounded claim. A covering sized by cost over an equal-area integer key is a sound basis for a sky index: on 10- and 50-million-row catalogues it is faster than pgSphere for cone searches at most radii warm and at every radius cold, by the largest margins where an index should matter most – large regions, cold caches, polygons – and it draws level with Q3C on cross-matching, the operation Q3C exists for. It does this with an index a

Table 11: A 20.5-million-row ObsCore-shaped relation under memory pressure: 768 MB of shared buffers against a 22 GB heap, 3500 uniformly drawn 2° cones covering the sphere about once. Each index is measured alone, with the server restarted between phases; the two medians are the two orderings of the phases.

	pgSphere	skycell
index size	1162 MB	439 MB
per row	59.5 B	22.5 B
$\times$ buffer pool	1.51	0.57
median query	10.4 / 13.4 ms	2.5 / 2.8 ms
buffer pages per query	1108	729
index accesses from disk	90.6%	47.4%
index resident	61 MB (5.2%)	33 MB (7.4%)

third the size, built thirty times faster, and with row estimates twice as accurate. The same conclusions hold on a density field taken from the real sky and on one designed by us, and the advantage on large regions roughly doubles between 10^7 and 5×10^7 rows, which is the evidence that the page count, and not something particular to one table size, produces it.

They do not establish that `skycell` is the better choice in general. Warm, between $6'$ and 1° , it is indistinguishable from pgSphere on one corpus and slightly slower at $6'$ on the other. On a 500 000-row ObsCore relation it is slower; on the small-cone ObsCore class it is slower in one cache regime and level in another, and we do not claim that row. Better row estimates did not, on the join shapes we wrote, produce better plans. And nothing here has run on real catalogue positions, on more than one machine, or with more than one storage device.

What remains expensive is planning. A covering is computed per plan, and even cached it costs 0.015–0.12 ms against 0.006–0.008 ms for a GiST predicate that needs no such step; that is the whole of `skycell`’s disadvantage at $6'$ and the reason small relations favour pgSphere. The architectural way out is to stop planning ranges: an SP-GiST operator class over the cell hierarchy would present a single index qualification and refine during the index descent, where the work is proportional to the rows actually present, removing both the per-plan covering and the per-arm planner cost our measurements put at 8 μ s for each range in the disjunction.

The density synopsis is the other limit. An equi-depth histogram cannot see a cluster smaller than one of its buckets, and because it is equi-depth, raising the statistics target does not help – a denser region gets narrower buckets, so interpolating inside one is scale invariant. The area guard bounds the damage; a custom `typanalyze` storing a multi-order count map would estimate what the covering actually needs. We have implemented such a map as an option and have no measurement showing it helps, which is why it is not part of the result above.

9 Conclusions

`skycell` keeps Q3C’s compact integer key and B-tree while answering the region queries `pgSphere` was built for, by treating the depth of a covering as a cost question whose answer follows from the local source density and the price of an index range.

Under a protocol designed to remove the advantages that measurement order and cache state hand to whichever method is measured last, on two 10-million-row corpora – one drawn from the real Gaia DR3 density field – it is 6–11% faster than `pgSphere` for cone radii of $1''$ – $1'$ and 31–37% faster at 3° warm, and faster at every radius with caches dropped, by a factor of up to 2.5. It answers convex polygons 2.7–4.4 times faster, keeps an index a third the size of `pgSphere`’s – 38 GiB against 160 GiB at Gaia DR3 scale for the pair of indexes that covers the same ground, which is the difference between a 64 GiB server and a 256 GiB one – builds it thirty times faster, and estimates row counts twice as accurately. On cross-matching, the operation Q3C exists for, it is Q3C’s equal (0.78–0.92 of its time, intervals reaching 1) and 2.5–3.3 times faster than `pgSphere` – so one index is competitive with each of the two established ones on the workload that each was built for. Its cost model’s one parameter is derived from the relation’s rows per page and the planner’s cost factors rather than configured – indistinguishable in speed from the constant it replaces, but no longer a number anyone has to choose – its chosen cell order costs about 3% against the best available one, measured with the two interleaved, and its density estimate is unbiased wherever a density is defined.

It is not the better choice everywhere, and scale sharpens the distinction rather than removing it. Raising the catalogue to 50 million rows widens the advantage for regions of $30'$ and above – to a factor 3.2 at 1° with cold caches – and leaves cones of an arcminute and below where they were, level with `pgSphere`, because there both methods touch a handful of pages at any table size and what remains is planning. On a small relation `skycell` is slower for that reason, and inside a TAP server whose cost is dominated by Python it changes nothing at all. On an ObsCore-shaped relation it leads the degree-scale classes at every size measured while the arcminute-scale class depends on the cache regime by a factor of two in either direction, which we do not claim.

An index is worth choosing carefully when the scan dominates. Which queries those are is a property of the archive, and we have tried to say precisely which ones they were here.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement

quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence. Both corpora are generated from seeds by the scripts recorded there – the Gaia corpus, at both scales, from the density map fetched from the ESA archive – so they are reproducible without transferring data; `bench/run.sh` and the numbered scripts 07–10 reproduce every table. The TAP A/B of Sect. 7 is recorded, protocol and results, in the same repository.

Acknowledgements. This work has made use of data from the European Space Agency (ESA) mission *Gaia* (<https://www.cosmos.esa.int/gaia>), processed by the *Gaia* Data Processing and Analysis Consortium (DPAC, <https://www.cosmos.esa.int/web/gaia/dpac/consortium>). Funding for the DPAC has been provided by national institutions, in particular the institutions participating in the *Gaia* Multilateral Agreement. Only per-cell source counts were used, to build the density field of Sect. 5.1.

The implementation, the benchmark harness and the analysis were developed with the assistance of Anthropic’s Claude Code. Because that bears on how the results should be read, we state what was done about it: the geometric predicates and the covering are checked by the brute-force suites of Sect. 4, which compare them against independent computations over millions of adversarially placed points rather than against expected outputs; every indexed answer in the regression suites is required to equal a sequential scan with the exact predicate; and the error described in Sect. 2.4 was found by those tests rather than by inspection, which is the argument for relying on them. The benchmark harness was audited by reproducing its headline comparison under a different protocol (Sect. 6.3), which is how the difference between the two protocols came to light.

References

- Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, Astronomical Data Analysis Software and Systems XIII, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225
- Delli Veneri, M., & Salgado, J. 2026, A&A, submitted
- Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, *Astron. Comput.*, 7–8, 27
- Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, IVOA Recommendation: Table Access Protocol Version 1.1
- ESA 1997, The Hipparcos and Tycho Catalogues, ESA SP-1200
- Fernique, P., Boch, T., Donaldson, T., et al. 2014, IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0

- Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets, Microsoft Research Technical Report MSR-TR-2006-52
- Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124
- Ortiz, I., Lusted, J., Dowler, P., et al. 2008, IVOA Recommendation: Astronomical Data Query Language Version 2.0
- Salgado, J., González-Núñez, J., Gutiérrez-Sánchez, R., et al. 2017, *Astron. Comput.*, 21, 22
- Salgado, J., et al. 2020, in ASP Conf. Ser. 522, *Astronomical Data Analysis Software and Systems XXVII*, ed. P. Ballester, J. Ibsen, M. Solar, & K. Shortridge (San Francisco: ASP), 307